LESSON 02 · API · 50 MIN

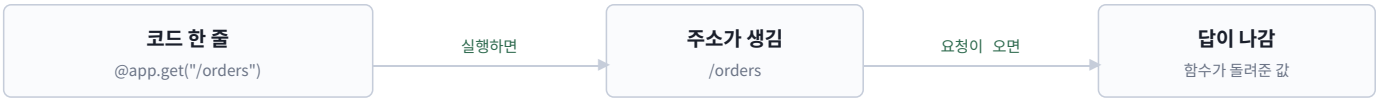
2교시 내 API 만들기 — 코드 한 줄이 주소 하나

1교시에는 남의 API를 불러 봤습니다. 이번엔 요청을 받아 답을 돌려주는 쪽을 직접 만듭니다. 새로 이해할 것은 딱 하나 — **함수 위에 붙인 한 줄이 주소를 만든다**는 것입니다.

설명 10분 · 따라하기 30분 · 정리와 질문 10분

OVERVIEW

내가 쓴 코드가 주소가 되기까지



> 함수 이름은 사람이 알아보려고 붙이는 것일 뿐입니다. **주소를 정하는 것은 함수 위의 한 줄**이고, **답이 되는 것은 함수가 돌려주는 값**입니다. 이 둘만 기억하면 됩니다.

WORDS

새로 나오는 말

말	무슨 뜻인가	언제 나오나
서버	요청을 기다리는 프로그램	내 컴퓨터에서 켜 둔다
uvicorn	서버를 켜는 명령	터미널에 입력
@app.get("/주소")	이 주소로 오면 아래 함수 실행	함수 바로 위에 붙인다
return	돌려줄 값	이게 그대로 응답이 된다
/docs	자동으로 만들어지는 설명서	버튼으로 직접 호출해 볼 수 있다
GET	보여줘	아무것도 바꾸지 않는다
POST	새로 만들어줘	누를 때마다 늘어난다
{order_id}	주소 안의 빈칸	아무 숫자나 들어올 수 있다

FILES

실습용 파일 — 그대로 옮겨 적으세요

main.py

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

ORDERS = {
    7: {"item": "참치김밥", "qty": 2},
    8: {"item": "라면", "qty": 1},
}


@app.get("/")
def hello():
    return {"message": "안녕하세요"}


@app.get("/orders")
def list_orders():
    return list(ORDERS.values())


@app.get("/orders/{order_id}")
def get_order(order_id: int):
    if order_id not in ORDERS:
        raise HTTPException(404, "그런 주문은 없습니다")
    return ORDERS[order_id]
```

STEPS

실습 순서

STEP 01 준비 확인

필요한 것이 깔려 있는지 봅니다.

```
python --version
# 안 되면 python3 --version 도 해보세요
```

확인 포인트

- [ ] 3.11 이상 숫자가 나온다

> 숫자가 안 나오고 오류가 뜨면 **손을 들어 주세요.** 여기서 막히면 오늘 하루가 힘들어집니다. 혼자 해결하려 하지 마세요.

STEP 02 폴더 만들고 준비하기

오늘 하루 쓸 작업 폴더를 만듭니다.

```
mkdir my-api
cd my-api
python -m venv .venv
# 맥/리눅스
source .venv/bin/activate
# 윈도우
.venv\Scripts\activate

pip install "fastapi[standard]" uvicorn
```

확인 포인트

- [ ] 프롬프트 맨 앞에 **(.venv)** 가 보인다
- [ ] 설치가 끝나고 오류 없이 프롬프트가 돌아온다

> 이 폴더를 **5교시까지 계속 씁니다.** 교시마다 새로 만들지 마세요.
> (.venv) 가 안 보이면 활성화가 안 된 것입니다. 위 명령을 다시 실행하세요.

STEP 03 가장 짧은 API 만들기

다섯 줄로 서버를 띄웁니다.

```
# main.py 파일을 만들고 아래만 적습니다
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def hello():
    return {"message": "안녕하세요"}

# 터미널에서
uvicorn main:app --reload
```

확인 포인트

- [ ] 터미널에 **Uvicorn running on http://127.0.0.1:8000** 이 보인다
- [ ] 브라우저에서 http://127.0.0.1:8000 을 열면 안녕하세요가 나온다

> 이 화면이 나오면 **여러분은 방금 서버를 만든 겁니다.** 1교시에 남의 서버를 불렀는데, 이제 남이 부를 수 있는 서버를 가진 거예요.

STEP 04 자동 설명서 열어 보기

따로 만들지 않아도 문서가 생긴다는 걸 봅니다.

```
# 브라우저에서
http://127.0.0.1:8000/docs
```

확인 포인트

- [ ] 초록색 GET 상자가 보인다
- [ ] **Try it out** → **Execute** 를 누르면 답이 나온다

> **오늘 실습은 대부분 이 화면에서 합니다.** curl 을 외울 필요 없이 버튼만 누르면 됩니다.

STEP 05 주문 목록 돌려주기

코드 한 줄이 주소 하나를 만드는 걸 확인합니다.

```
# 위 '실습용 파일'의 main.py 전체로 바꿔 넣습니다
# (--reload 때문에 저장하면 자동으로 다시 켜집니다)

# 브라우저에서 차례로 열어 보세요
http://127.0.0.1:8000/
http://127.0.0.1:8000/orders
```

확인 포인트

- [ ] / 는 안녕하세요, /orders 는 주문 목록이 나온다
- [ ] 주문 목록은 **대괄호** 로 감싸져 있다 (여러 개니까)

> 함수는 두 개, 주소도 두 개입니다. **함수 위에 붙인 줄이 주소를 정합니다.** 함수 이름(hello, list_orders)은 주소와 아무 상관이 없어요.

STEP 06 주문 하나만 보기 + 404 만들기

주소의 빈칸이 함수 입력이 되는 걸 봅니다.

```
http://127.0.0.1:8000/orders/7
http://127.0.0.1:8000/orders/8
http://127.0.0.1:8000/orders/999
http://127.0.0.1:8000/order
```

확인 포인트

- [ ] 7과 8은 주문 하나가 **중괄호** 로 나온다
- [ ] 999는 404 — 주소는 맞는데 그 주문이 없어서
- [ ] /order 도 404 — 그런 주소를 만든 적이 없어서

> **같은 404인데 원인이 다릅니다.** 하나는 물건이 없는 것, 하나는 주소를 잘못 적은 것.
> 실습 중 404가 나오면 **주소 오타부터 확인**하세요. 가장 흔한 원인입니다.

STEP 07 POST로 새로 만들기

보여주는 것과 바꾸는 것의 차이를 느낍니다.

```
# main.py 에 추가
from pydantic import BaseModel

class OrderIn(BaseModel):
    item: str
    qty: int

@app.post("/orders")
def create_order(order: OrderIn):
    new_id = max(ORDERS) + 1
    ORDERS[new_id] = {"item": order.item, "qty": order.qty}
    return ORDERS[new_id]

# /docs 에서 POST 상자 → Try it out → Execute
```

확인 포인트

- [ ] POST 를 한 번 누르면 주문이 하나 늘어난다
- [ ] /orders 를 다시 열어 확인해 보세요
- [ ] POST 를 세 번 누르면 **세 개** 가 늘어난다

> **GET 은 몇 번을 눌러도 그대로, POST 는 누를 때마다 늘어납니다.**
> 그래서 인터넷 쇼핑에서 결제 버튼을 두 번 누르면 두 번 주문될 수 있는 거예요.

STEP 08 오늘 만든 것 확인하고 정리

무엇을 만들었는지 스스로 정리합니다.

```
# /docs 를 다시 열어 만든 주소를 세어 봅니다
GET  /
GET  /orders
GET  /orders/{order_id}
POST /orders

# 서버는 3교시에도 쓰니 끄지 말고 두세요 (Ctrl+C 로 끕니다)
```

확인 포인트

- [ ] 주소 네 개가 보인다
- [ ] 각각 무슨 일을 하는지 말할 수 있다

> 주소 네 개짜리 API를 만들었습니다. 실제 서비스도 이게 수십 개로 늘어난 것뿐이에요. **구조는 똑같습니다.**

WHEN STUCK

막혔을 때 보는 곳

증상	원인	해결
`(.venv)` 가 안 보임	가상환경 활성화가 안 됨	활성화 명령을 다시 실행 · 터미널을 새로 열었다면 매번 필요
`Address already in use`	8000 포트를 이미 쓰는 중	uvicorn main:app --reload --port 8001 로 실행
코드를 고쳤는데 안 바뀜	--reload 없이 켰거나 저장을 안 함	파일 저장 확인 후 Ctrl+C → 다시 실행
`IndentationError`	파이썬 들여쓰기 오류	함수 안쪽은 스페이스 4칸 · 탭과 섞어 쓰지 않기
`404`가 계속 나옴	주소 오타	/orders 인지 /order 인지 확인 · /docs 에 있는 주소와 비교

CHECK

스스로 확인하기 — 답할 수 있으면 통과

- [ ] 함수 위에 붙이는 한 줄은 무엇을 정하나요?
- [ ] 함수가 return 한 값은 어디로 가나요?
- [ ] /orders 와 /orders/7 은 무엇이 다른가요?
- [ ] GET 과 POST 를 각각 한 문장으로 설명해 보세요.
- [ ] 404가 나왔을 때 가장 먼저 확인할 것은?

한 줄 정리

함수 위의 한 줄이 주소를 만들고, 함수가 돌려주는 값이 답이 됩니다. API 만들기는 이게 전부입니다.

다음 시간 → 3교시 — 새로고침 없이 화면이 바뀌게 하기

WORKSHEET

막힌 STEP 번호 · 화면에 나온 빨간 글씨 · 강사에게 물어볼 것